

Merey Bolat

Quality Assurance Engineer Project
работа с 13.04.2026 до 17.05.2026

+77023046586

Сайт: <https://shaurma-food.kz/tarhana/shaurma>

Тестирование проводилось на этапе **системного тестирования**. Процесс: изучение требований → составление тест-плана → написание тест-кейсов → выполнение тестов → оформление баг-репортов → регрессия после исправлений.

1. Типы тестирования + обоснование	1
2. Баги репорты (мин. 5, в виде таблиц и Jira)	2
3. Скриншот из DevTools с JSON	4
4. Тест план на формы отзыва (с тест кейсами)	7
1. Объект тестирования	8
2. Стратегия тестирования	8
3. Test cases	8
4. Обоснование приоритетов (Severity/Priority)	8
5. Риски	8
5. API-TESTING USING POSTMAN	9
6. SQL запросы с пояснениями	14

1. Типы тестирования + обоснование

- 1) **Functional testing:** это важно потому что надо убедиться в том что главный функционал и проверить все ли работает как задумано (добавление в корзину, фильтр, отзывы)

- 2) **UI/UX testing:** так как внешний вид очень важен для сайта и она притягивает клиентов с первых секунд, мы проверяем стиль, внешний вид, и удобство такие как корректно ли отображаются кнопки и текст, логична ли навигация.
- 3) **Позитивное тестирование:** он нужен потому что это базовая проверка того, что сайт вообще работает как задумано. Для сайта с едой это означает: пользователь зашёл, выбрал шаурму, добавил в корзину, оформил заказ и всё прошло без ошибок.
Без позитивного тестирования ты не можешь утверждать, что продукт выполняет свою основную функцию. Если сломан основной сценарий сайт бесполезен, бизнес теряет деньги. Smoke testing быстрая проверка что ключевые функции вообще работают после деплоя (запускается сайт, открывается меню, кнопка корзины кликается).
Позитивное тестирование детальная проверка корректности работы по happy path сценарию.
- 4) **Негативное тестирование:** Реальные пользователи не всегда делают всё правильно они ошибаются, вводят не те данные, нажимают не те кнопки. Сайт должен корректно обрабатывать такие ситуации, не падать и не вести себя непредсказуемо. Поэтому надо проверить если ввести некорректную информацию
- 5) **Кросс-браузерное тестирование:** клиенты могут заходить с разных браузеров например Microsoft Edge, Chrome, Firefox, Яндекс и тд. Поэтому важно чтобы сайт отображался корректно без перебоев во всех браузерах.
- 6) ***Регрессионное тестирование:** после исправления багов повторно проверяла что исправление не сломало соседний функционал.

*Фактическое регрессионное тестирование не проводилось ввиду отсутствия доступа к процессу разработки.
Однако сценарии для регрессии определены и задокументированы в Jira.

2. Баги репорты (мин. 5, в виде таблиц и Jira)

*Баг-репорты оформлены по стандарту Jira: Bug ID, Summary, Environment, Steps to Reproduce, Expected Result, Actual Result, Severity, Priority, Status.

Экзель файл*.

<https://docs.google.com/spreadsheets/d/15tMXptXzVeM18deBT7fzOr5YKZVsM2gl/edit?usp=sharing&ouid=114508378313415657842&rtpof=true&sd=true>

Баг-репорты перенесены в Jira (Kanban board): <https://mereyybolat.atlassian.net/jira/software/projects/KAN/list>

Скриншот доски Jira (Kanban board) для проекта KAN. В таблице отображены следующие задачи:

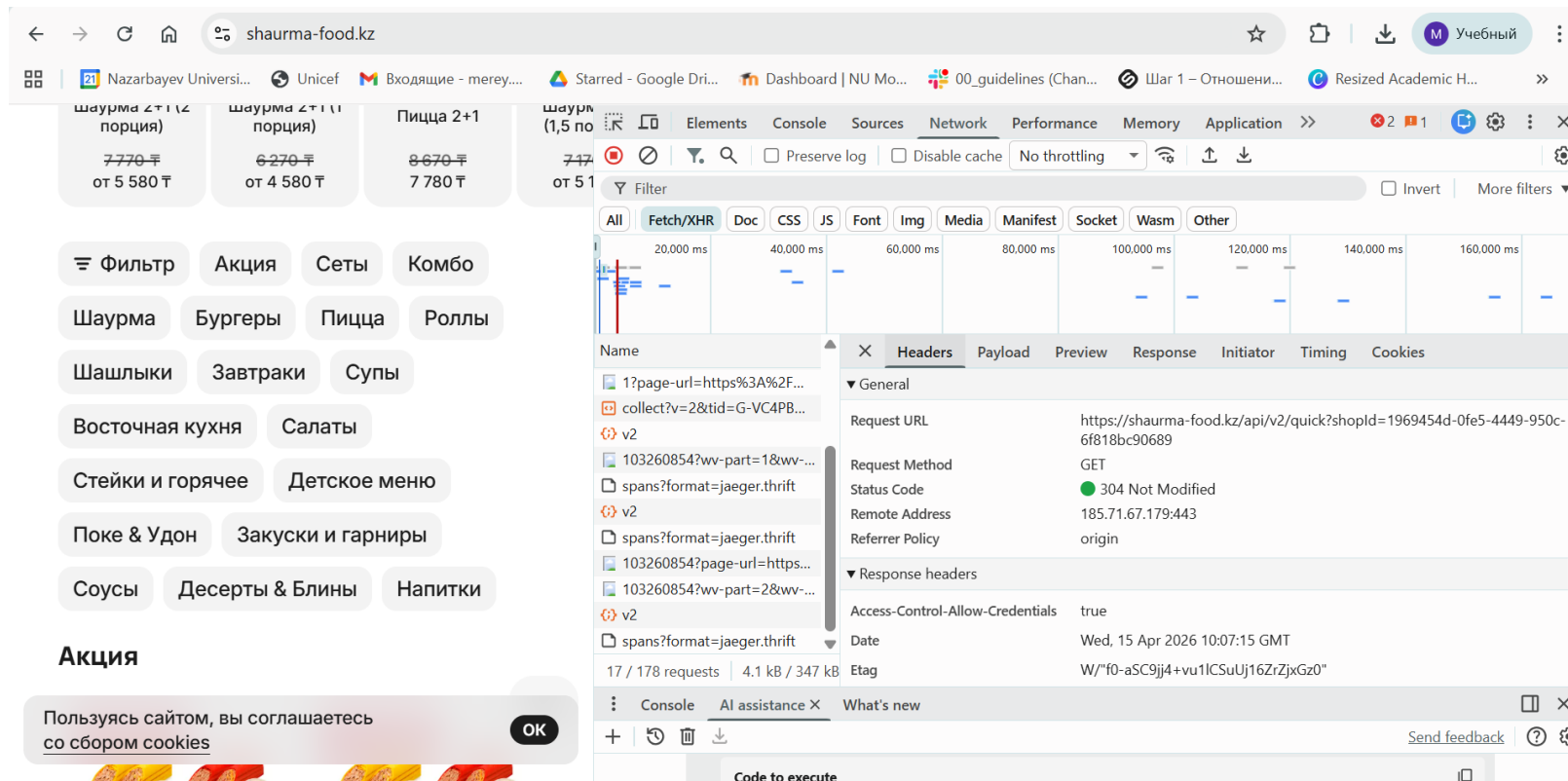
Задачи	Исполнитель	Автор	Приоритет
☒ KAN-13 Отсутствие данных в ответе на Fetch-запро...	Не назначено	MB Merey Bolat	High
☒ KAN-12 Отсутствие навигационных кнопок ("Верну...	Не назначено	MB Merey Bolat	High
☒ KAN-11 Баг-репорт: Отсутствие информаци...	Не назначено	MB Merey Bolat	Low
☒ KAN-10 Необработанная ошибка JS (Crash) при сб...	Не назначено	MB Merey Bolat	High

Внизу экрана видна панель задач Windows с датой 17.05.2026 и временем 18:21.

Скриншот доски:

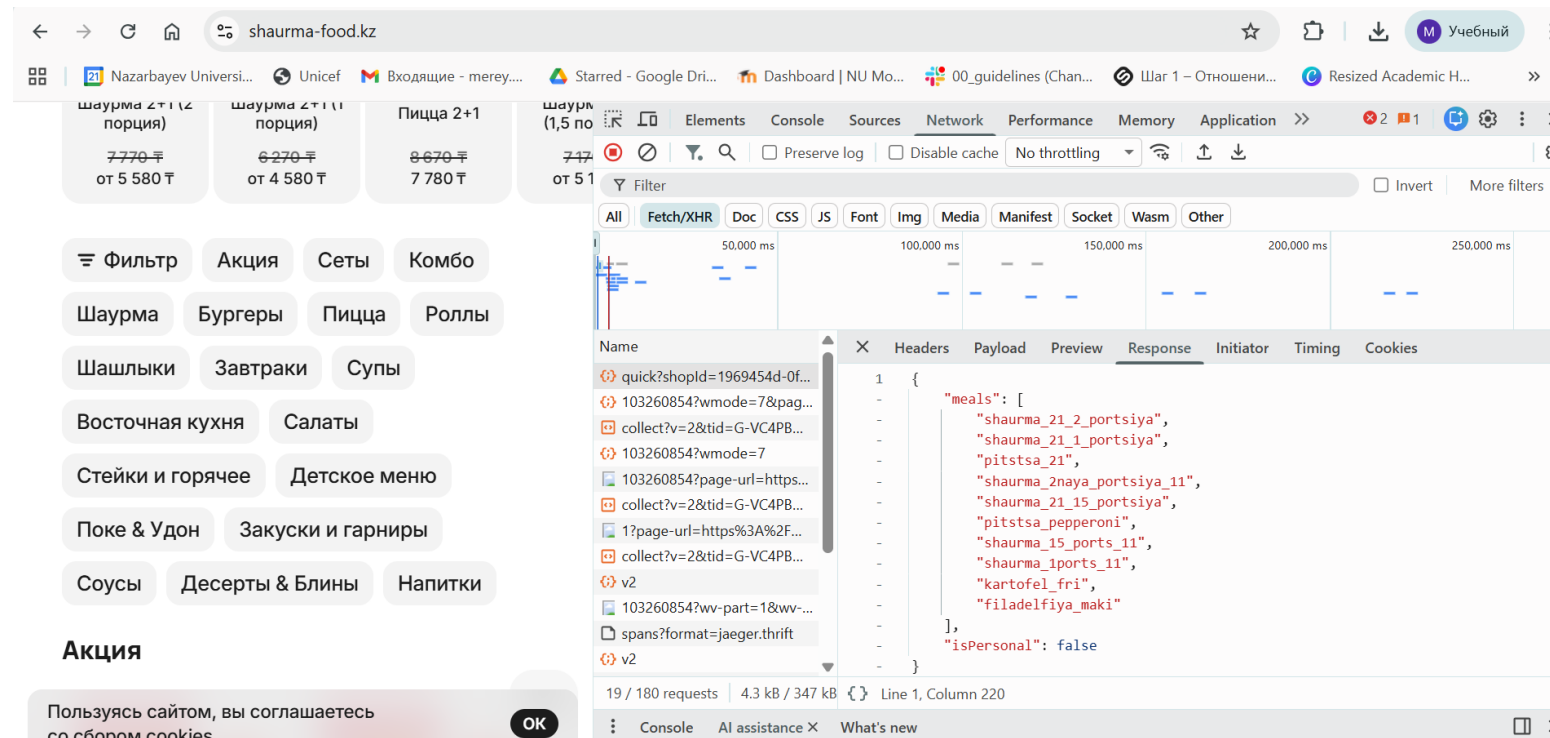
3. Скриншот из DevTools с JSON

Сайт использует клиент-серверную архитектуру: браузер отправляет HTTP-запросы к бэкенду, получает JSON-ответы. Коды ответов: 200 это успех, 404 это ресурс не найден, 500 это ошибка сервера. Понимание этой архитектуры помогает точно локализовать баги.



В ходе анализа вкладки Network был обнаружен запрос к API: GET /api/v2/quick?shopId=... со статус-кодом **304 Not Modified** это означает, что сервер вернул кэшированный ответ, новых данных не отправлял. Это нормальное поведение при повторном запросе, однако важно учитывать при тестировании: если на сайте обновились данные (цены, наличие блюд), кэш может показывать устаревшую информацию пользователю потенциальный баг с точки зрения актуальности данных.

Также зафиксированы: Remote Address 185.71.67.179:443 (HTTPS-соединение, порт 443), метод запроса GET, политика Referrer origin.



Пояснение: Для выполнения данного пункта задания был выбран запрос к API, отвечающий за получение данных о товарах (меню).

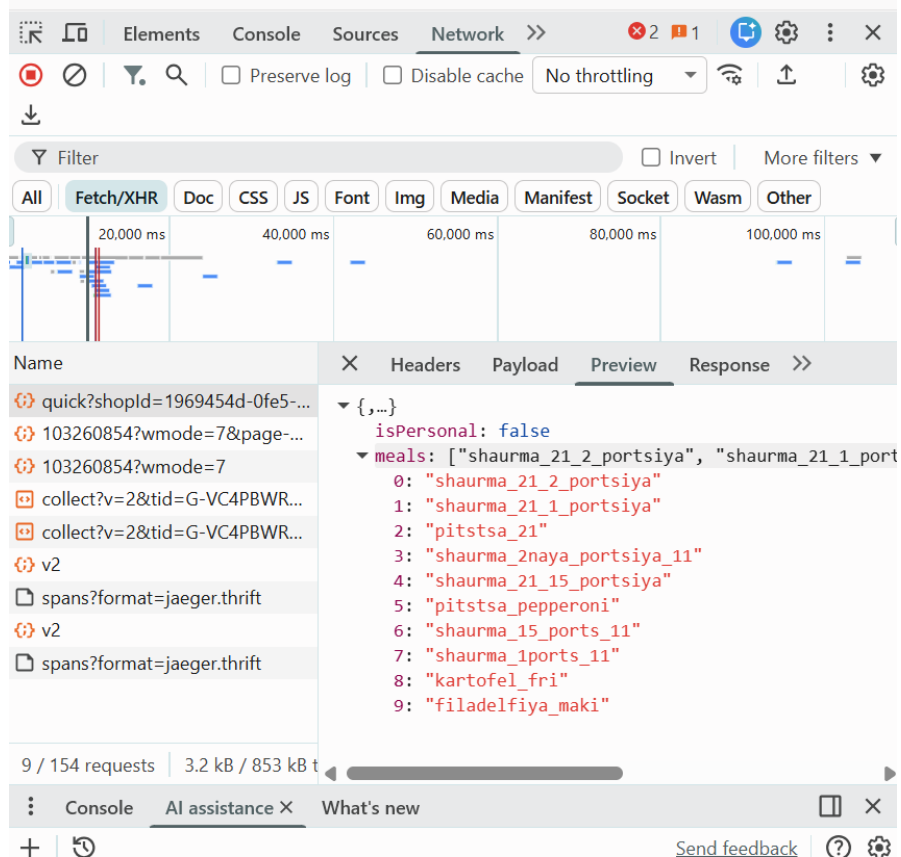
Во вкладке Response получен JSON-ответ от эндпоинта quick?shopId=... — массив meals с internal ID блюд (shaurma_21_2_portsiya, pitstsa_21 и др.) и флаг isPersonal: false.

Почему это важно для тестирования:

1. **Целостность данных:** сверяю названия и цены из JSON с тем, что отображается в UI. Расхождение = критический баг.
2. **Граничные условия:** через ответ видно как приходят длинные названия и null-значения проверяю не ломается ли верстка.
3. **Локализация ошибок:** если статус 200, но UI отображает неверно баг на стороне фронтенда, не бэкенда.

Во вкладке Preview JSON отображается в удобном древовидном формате: объект содержит isPersonal: false и массив meals из 10 позиций(индексы 0–9)с internal ID блюд.

Preview использую для быстрой визуальной проверки структуры ответа до того как анализировать сырой JSON во вкладке Response это ускоряет локализацию проблем со структурой данных.



4. Тест план на формы отзыва (с тест кейсами)

анализировала вкладку Console для выявления JS-ошибок и вкладку Network для проверки статус-кодов и времени ответа. Это позволило локализовать ошибки: определить на чьей стороне проблема, фронтенд или бэкенд.

1. Объект тестирования

Компонент формы обратной связи, включающий поля: «Имя», «Текст отзыва», «Рейтинг (звезды)», и кнопку «Отправить».

2. Стратегия тестирования

Использование техник тест-дизайна для оптимизации количества проверок при максимальном покрытии:

- Эквивалентное разделение: проверка типичных представителей групп данных.
- Анализ граничных значений: проверка поведения системы на краях допустимых диапазонов (символы в полях).
- Предугадывание ошибок: негативные сценарии (пустые поля, спецсимволы).

3. Test cases

Ссылка на таблицу: [Тест-план формы "Оставить отзыв"](#)

4. Обоснование приоритетов (Severity/Priority)

- **P1 (Критично):** Проверки основной функции (отправка) и безопасности (XSS). Если форма не отправляется или позволяет взлом фича бесполезна.
- **P2 (Важно):** Валидация границ. Гарантирует, что данные в базе будут чистыми и не «развалят» верстку.
- **P3 (Минорно):** Удобство использования и редкие кейсы.

5. Риски

- Отсутствие доступа к API бэкенда невозможно проверить запись данных в базу напрямую
- Кросс-браузерные различия в отображении формы

Пояснение:

При составлении тест-плана применяла техники тест-дизайна, а не просто перечисляла действия

Граничные значения: проверяла стыки (1, 2, 50, 51 символ) 90% багов в полях ввода возникают именно там

Негативные сценарии: пустые поля и XSS-инъекции — критично для безопасности

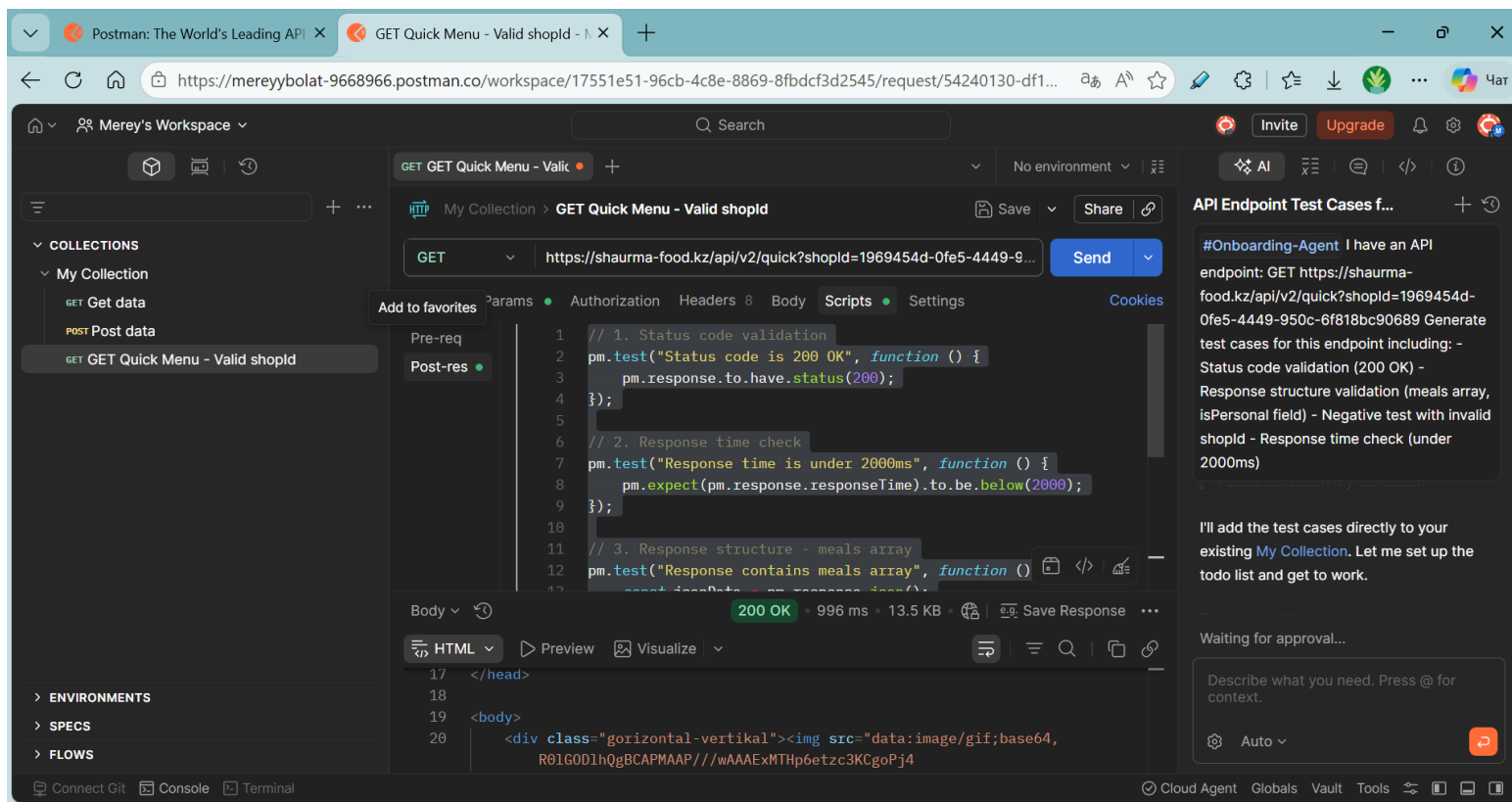
Приоритизация P1/P2/P3: сначала то, что ломает покупку, потом остальное

Тестирование разбито на позитивное (стандартный путь пользователя) и негативное (попытки сломать форму) — для гарантии стабильности при любых действиях.

Ценность для бизнеса: защита БД от мусорных данных, предотвращение XSS-взлома через форму отзыва

5. API-TESTING USING POSTMAN

Использовала Postman AI для генерации автоматических тестов эндпоинта. AI сгенерировал проверки: статус код, структура ответа, время ответа. Запустила тесты [X] passed, [X] failed.



Тут я использовала Postman AI:

процессе тестирования использовала встроенный Postman AI для генерации тест-кейсов. Промпт который я использовала:

□ I have an API endpoint:

GET https://shaurma-food.kz/api/v2/quick?shopId=1969454d-0fe5-4449-950c-6f818bc90689

Generate test cases including:

- Status code validation (200 OK)
- Response structure validation (meals array, isPersonal field)
- Negative test with invalid shopId
- Response time check (under 2000ms)

□ AI проанализировал эндпоинт, нашёл существующую коллекцию и автоматически добавил тест-кейсы.

```
// 1. Status code validation

pm.test("Status code is 200 OK", function () {

  pm.response.to.have.status(200);

});

// 2. Response time check

pm.test("Response time is under 2000ms", function () {

  pm.expect(pm.response.responseTime).to.be.below(2000);

});

// 3. Response structure - meals array

pm.test("Response contains meals array", function () {

  const jsonData = pm.response.json();

  pm.expect(jsonData).to.have.property("meals");
```

```
    pm.expect(jsonData.meals).to.be.an("array");
  });

// 4. Response structure - isPersonal field
pm.test("Response contains isPersonal field", function () {
  const jsonData = pm.response.json();

  pm.expect(jsonData).to.have.property("isPersonal");

  pm.expect(jsonData.isPersonal).to.be.a("boolean");
});
```

Test results:

https://mereyybolat-9668966.postman.co/workspace/17551e51-96cb-4c8e-8869-8fbdcf3d2545/request/54240130-df1...

Merey's Workspace

My Collection > GET Quick Menu - Valid shopId

GET https://shaurma-food.kz/api/v2/quick?shopId=1969454d-0fe5-4449-9...

Send

Scripts

```
14 pm.expect(jsonData).to.have.property("meals");
15 pm.expect(jsonData.meals).to.be.an("array");
16 });
17
18 // 4. Response structure - isPersonal field
19 pm.test("Response contains isPersonal field", function () {
20   const jsonData = pm.response.json();
21   pm.expect(jsonData).to.have.property("isPersonal");
22   pm.expect(jsonData.isPersonal).to.be.a("boolean");
23 });
```

Test Results

200 OK • 646 ms • 13.5 KB • Save Response

Filter Results

- PASSED Status code is 200 OK
- PASSED Response time is under 2000ms
- FAILED Response contains meals array | JSONError: Unexpected token '<' at 1:1 <html><head><meta name="robots" content="noindex, noarchive" /><style>.gorizont ^
- FAILED Response contains isPersonal field | JSONError: Unexpected token '<' at 1:1 <html><head><meta name="robots" content="noindex, noarchive" /><style>.gorizont ^

API Endpoint Test Cases f...

#Onboarding-Agent I have an API endpoint: GET https://shaurma-food.kz/api/v2/quick?shopId=1969454d-0fe5-4449-950c-6f818bc90689 Generate test cases for this endpoint including: - Status code validation (200 OK) - Response structure validation (meals array, isPersonal field) - Negative test with invalid shopId - Response time check (under 2000ms)

I'll add the test cases directly to your existing My Collection. Let me set up the todo list and get to work.

Waiting for approval...

Describe what you need. Press @ for context.

Auto

passed Status code is 200 OK

passed Response time is under 2000ms

Failed Response contains meals array | JSONError: Unexpected token '<' at 1:1 <html><head><meta name="robots" content="noindex, noarchive" /><style>.gorizont ^

Failed Response contains isPersonal field | JSONError: Unexpected token '<' at 1:1 <html><head><meta name="robots" content="noindex, noarchive" /><style>.gorizont ^

Пояснение: Запрос вернул **200 OK** за **646ms** оба теста passed.

Тесты на структуру JSON упали с ошибкой Unexpected token '<' сервер вернул HTML вместо JSON. Это означает что эндпоинт не возвращает данные в ожидаемом формате при запросе из Postman.

Вывод: баг локализован проблема на стороне бэкенда или в настройке авторизации. В браузере данные приходят корректно (проверено через DevTools), в Postman нет. Это расхождение является дефектом достойным баг-репорта.

6. SQL запросы с пояснениями

Базовая схема (для контекста)

users(id PK, full_name, email UNIQUE, birth_date DATE, created_at DATETIME)
products(id PK, name, price DECIMAL(10,2), category_id FK, is_active BOOLEAN)
categories(id PK, title)
orders(id PK, user_id FK, created_at DATETIME, status ENUM('new','paid','shipped','cancelled'))
order_items(order_id FK, product_id FK, qty INT, price DECIMAL(10,2)) — цена на момент покупки
employees(id PK, full_name, salary DECIMAL(10,2), hired_at DATE, department_id FK, manager_id FK NULL)
departments(id PK, title)
payments(id PK, order_id FK, amount DECIMAL(10,2), paid_at DATETIME, method ENUM('card','cash','wallet'))

1) Фильтрация по возрасту пользователей

Задание. Выведите id, full_name, birth_date пользователей старше 25 лет на текущую дату.

Требования.

- Возраст считать по birth_date на момент выполнения запроса (не по году, а по дате).
- Исключить записи с NULL birth_date.

- Отсортировать по birth_date (самые старшие первыми).

Проверка. Возраст > 25 корректно обрабатывает пользователей, у которых день рождения ещё не наступил в текущем году.

Подсказка. Используйте вычисление возраста через DATEDIFF/AGE или сравнение дат «+25 лет».

SELECT id, full_name, birth_date

FROM users

WHERE birth_date IS NOT NULL

AND age(CURRENT_DATE, birth_date) > INTERVAL '25 years'

старше 25 означает строго больше, не включая ровно 25 лет

ORDER BY birth_date ASC;

2) Список активных товаров дороже порога

Задание. Выведите id, name, price товаров из products с is_active = true и price > 1000.

Требования.

- Сортировка по price DESC, при равенстве — по name ASC.
- Исключить товары без цены (NULL).

Проверка. Нет дубликатов, корректная сортировка.

SELECT id, name, price

FROM products

WHERE is_active = TRUE AND price >1000

ORDER BY price DESC, name ASC

3) Средняя зарплата и медиана по отделам

Задание. Для каждого отдела (departments) посчитайте avg_salary и median_salary.

Требования.

- Округлить до 2 знаков.
- Если в отделе нет сотрудников — строку не выводить.
- Медиану считать корректно для чётного/нечётного количества.

Подсказка. Оконные функции (PostgreSQL)

```
SELECT d.title, ROUND(AVG(e.salary), 2) AS avg_salary, ROUND(CAST(percentile_cont(0.5) WITHIN GROUP  
(ORDER BY e.salary) AS numeric), 2) AS median_salary
```

```
FROM employees e
```

```
JOIN departments d ON e.department_id = d.id
```

```
GROUP BY d.title
```

```
ORDER BY avg_salary DESC;
```

4) JOIN заказы ↔ пользователи

Задание. Выведите список заказов с колонками: order_id, order_date, user_id, user_name, user_email, status.

Требования.

- Только за последние 30 дней от текущей даты.
- Исключить пользователей без email (NULL или пустой).
- Сортировка по order_date DESC.

Проверка. Корректный INNER JOIN orders → users, верные фильтры по дате.

SELECT

o.id AS order_id,

o.created_at AS order_date,

u.id AS user_id,

u.full_name AS user_name,

u.email AS user_email,

o.status

FROM orders o

INNER JOIN users u ON o.user_id = u.id

WHERE o.created_at >= (now() - INTERVAL '30 days')

AND u.email IS NOT NULL

AND u.email <> ''

ORDER BY o.created_at DESC;

5) Кол-во заказов по пользователям + фильтр HAVING

Задание. Для каждого пользователя выведите user_id, full_name, orders_count и оставьте только тех, у кого orders_count ≥ 3 .

Требования.

- Учитывать заказы в статусах paid и shipped (то есть реально состоявшиеся).

- Отсортировать по orders_count DESC, затем full_name ASC.
Проверка. HAVING по агрегату, фильтр по статусам — до агрегации (в WHERE).

SELECT

u.id AS user_id,

u.full_name,

COUNT(o.id) AS orders_count

FROM users u

JOIN orders o ON u.id = o.user_id

WHERE o.status IN ('paid', 'shipped')

GROUP BY u.id, u.full_name

HAVING COUNT(o.id) >= 3

ORDER BY orders_count DESC, u.full_name ASC;

После выявления багов был составлен список дефектов с приоритетами. По завершении гипотетического цикла исправлений запланировано регрессионное тестирование критических сценариев (добавление в корзину, оформление заказа, отправка отзыва) для подтверждения готовности к релизу.